



Plateforme Microsoft .NET et Langage C#

Pr. Abderrahim BENABBOU

abderrahim.benabbou@gmail.com



Partie I

La plateforme Microsoft .NET

Build it with .NET

The diagram consists of four rounded rectangular boxes arranged horizontally. Each box has a colored border and a small icon at the top left. The first box is purple and contains a globe icon, the word "Web", and text about building web apps for macOS, Windows, Linux, and Docker. The second box is pink and contains a smartphone icon, the words "Mobile and desktop", and text about using a single codebase to build native apps for Windows, macOS, iOS, and Android. The third box is blue and contains a cloud icon, the word "Cloud", and text about building scalable and resilient cloud-native apps that run on all major cloud providers. The fourth box is purple and contains a microservices icon (four small squares), the word "Microservices", and text about creating independently deployable microservices that run on Docker containers. The background of the diagram features a faint image of a laptop keyboard and some abstract shapes.

- Web**
Build web apps and services for macOS, Windows, Linux, and Docker.
- Mobile and desktop**
Use a single codebase to build native apps for Windows, macOS, iOS, and Android.
- Cloud**
Build scalable and resilient cloud-native apps that run on all major cloud providers.
- Microservices**
Create independently deployable microservices that run on Docker containers.

Historique

- Le **.NET** est une plateforme de développement de logiciels, créée par MicroSoft en **2001** sous le nom **.Net Framework**.
- La version stable de .Net Framework 1.0 a été publiée en **2002**.
- Le .Net Framework était réservé au développement de logiciels sous Windows.
- La dernière version du .Net Framework est **.Net Framework 4.8**
- En 2014, Microsoft a commencé à écrire un successeur multiplateforme et open source pour .NET Framework. : le **.NET Core**
- A partir de la version **.NET Core 3.1**, la nomination a été simplifiée en **.NET 5** pour marquer que .Net Core est devenue la principale implémentation de .Net
- La version LTS actuelle (2024) est **.Net 8 (C# 12)**

Présentation

- **.Net est une plate-forme** de développement de logiciels
 - .Net fournit l'ensemble des technologies, langages et APIs nécessaires à la création de différents types d'applications
- **.Net open source et libre (Licence MIT)**
- **.Net multi plateformes**
 - .Net cible différentes plateformes : Windows, Linux, MacOS, Android, iOS, tvOS et watchOS
- **.Net multi langages**
 - .Net est multi langages (VB, VC++, Delphi, JavaScript, Python, Perl, etc) mais C# est le langage le plus utilisé dans la plateforme

Présentation

- **SDK .Net**

Le .NET offre un Kit de développement logiciel (SDK) comprenant un ensemble de bibliothèques (APIs) et d'outils permettant de développer et d'exécuter des applications .Net.

- **NuGet**

NuGet est un gestionnaire de packages spécialement conçu pour .NET qui contenant des milliers de packages qui étendent les APIs de .Net.

- **Technologies standards**

Le .NET prend totalement en charge les technologies Internet indépendantes des plates formes tels que HTTP, XML, Json, SOAP, WSDL, etc.

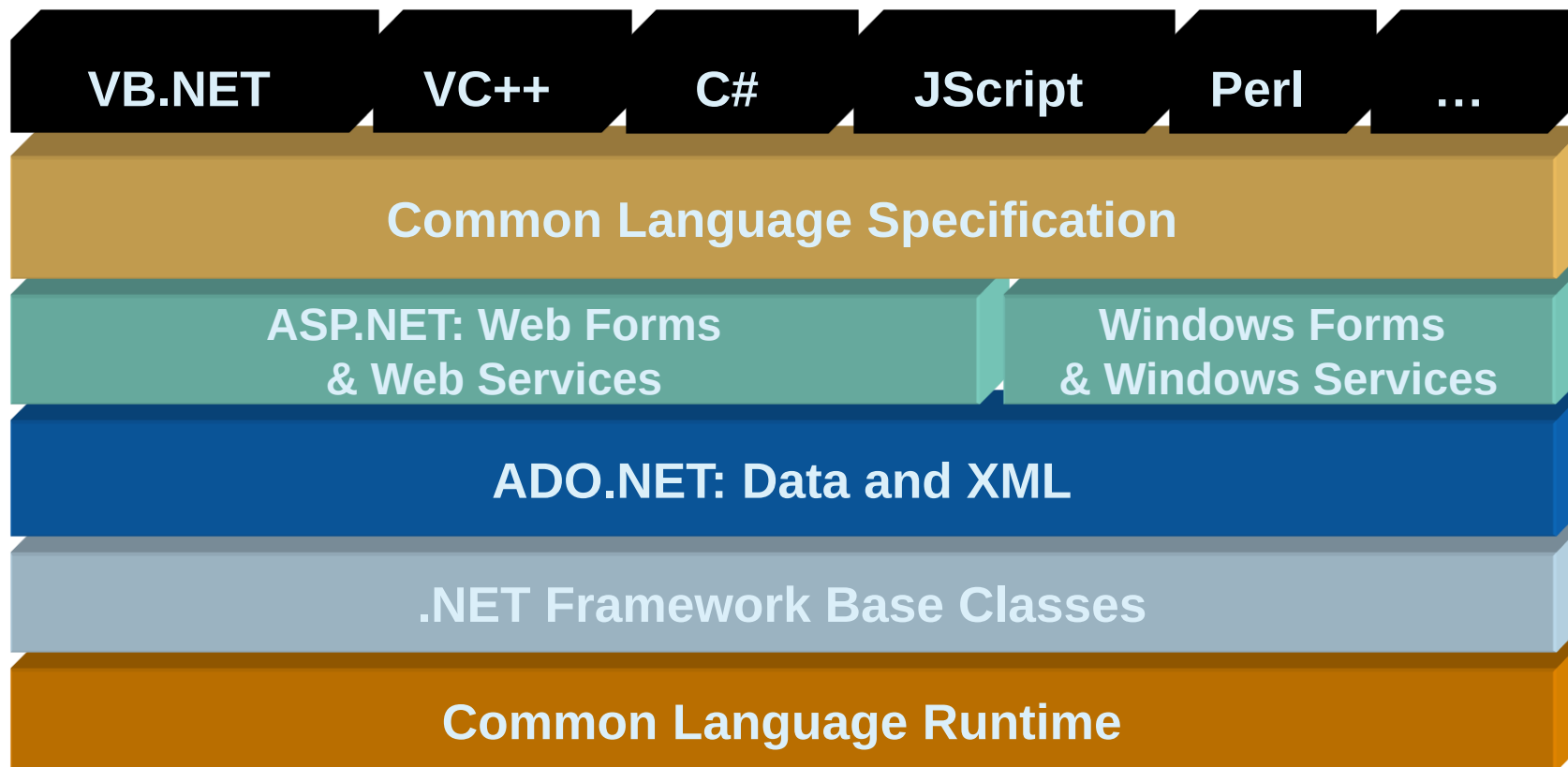
.NET Core

.NET Core est une implémentation .NET multiplateforme pour les applications bureaux, Web, Web distribuées, mobiles, applications dans le cloud, l'IA, des jeux et des IoT



Architecture .Net

- .NET possède une architecture structurée en plusieurs couches



Architecture de .NET

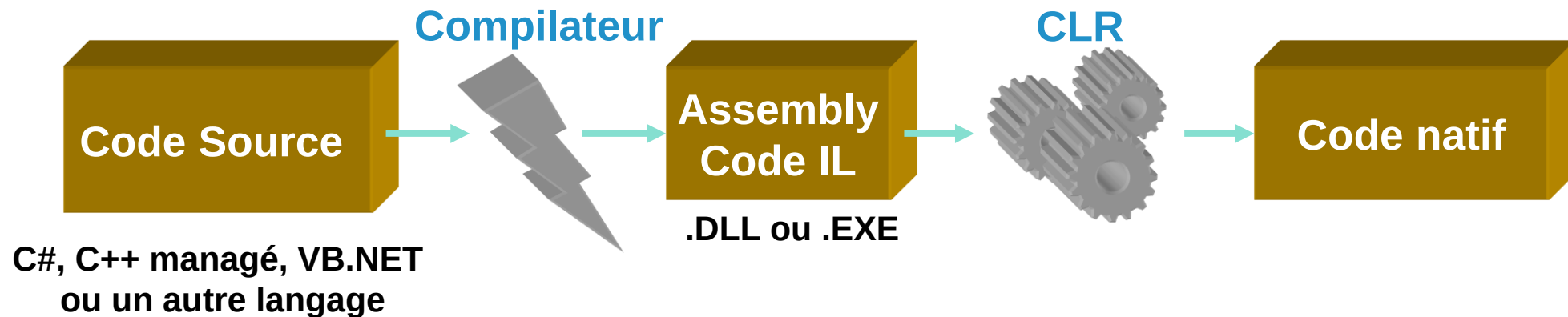
Common Language Runtime (CLR)

- Le **CLR** est l'environnement d'exécution du .NET Framework
- Le **CLR** est la machine virtuelle de Microsoft
- Le **CLR** est indépendant de l'architecture physique de la machine
- Le **CLR** exécute un bytecode écrit dans un langage intermédiaire nommé **MSIL** (Microsoft Intermediate Language) ou simplement **IL**
- Les tâches essentielles de **CLR** sont :
 - le chargement des classes,
 - la vérification de types,
 - la gestion de la mémoire (Garbage Collector), des exceptions et de la sécurité,
 - l'exécution du code IL

Architecture de .NET

Common Language Runtime (CLR)

- L'exécution par le **CLR** se traduit par une compilation **JIT (Just-In-Time)** : traduction à la volée du **code IL** appelé aussi **code managé**, en **code natif**



- Bien que se terminant par l'extension **.exe**, un programme issu d'une compilation sous **.NET** n'est pas un exécutable en code natif, mais un bytecode en **IL**
- Le **CLR** peut aussi être hébergé par des composants non managés qui peuvent le charger dans leurs processus et lancer l'exécution d'un code managé (exemple **l'Internet Explorateur**)

Architecture de .NET

Common Language Runtime (CLR)

- **Assembly**

C'est une unité logique de déploiement (un archive) auto descriptive qui contient :

- **Le Manifest** : un fichier de description de l'assemblage qui contient des Metadata décrivant les composants présents dans un assembly (version, types, dépendances, etc.)
- **Les Metadata Types** : Définition complète de tous les types présent dans l'assembly : attributs, méthodes, paramètres, ressources, etc.
- **Le Code IL** : le code managé
- **Les Ressources** : fichiers xml, bmp, jpg, etc. utilisés par l'assembly

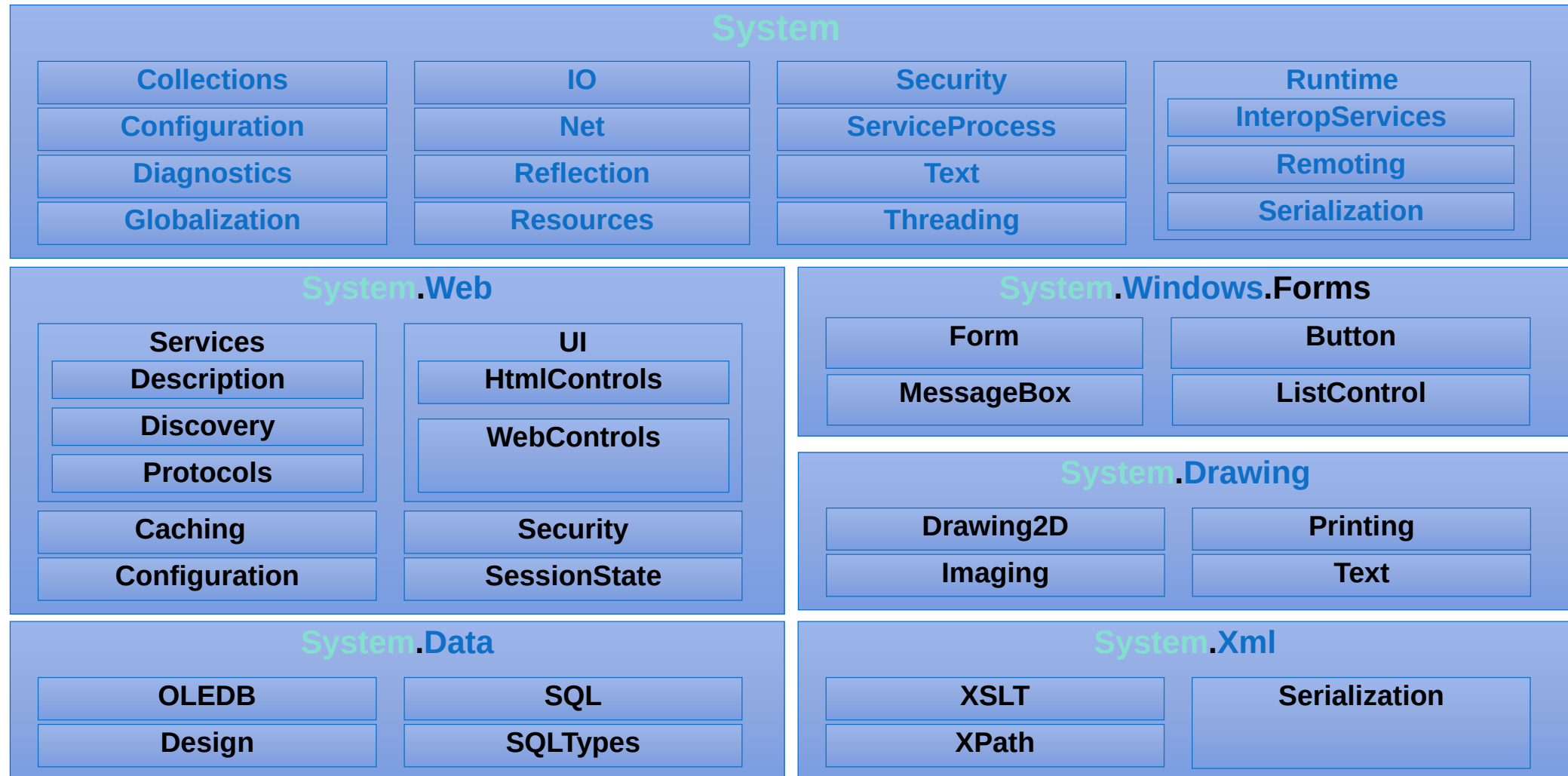
Architecture de .NET

.NET Framework Base Classes

- C'est un ensemble complet de classes permettant de développer :
 - des applications de bureau
 - Applications consoles
 - Applications à interface graphique utilisateur (GUI, *Graphical User Interface*) : les Windows Forms
 - des applications web qui utilisent l'ASP.NET et exploitent les services Web XML et les Web Forms
 - des applications hébergées ou à base de scripts
- Ces classes sont accessibles par tous les langages de la plate forme .NET et de tout langage conforme à la norme **CLS** (Common Language Specification)
- Les classes les plus couramment utilisées sont regroupées dans l'espace de noms nommé : **System**

Architecture de .NET

.NET Framework Base Classes



Architecture de .NET

.NET Framework Base Classes

•Exemples de classes

- **System** : types de base (Object, Byte, Char, etc.) et la classe Console
- **System.Collections** : Collections d'objets (ArrayList, Dictionary, SortedList, Stack, etc.)
- **System.Data** : Interaction avec les bases de données (DataRow, DataSet, DataTable, DataSource, etc.)
- **System.IO** : Accès aux fichiers (File, FileStream, StreamReader, StreamWriter, etc.)

Architecture de .NET

.NET Framework Base Classes

- **System.Net** : Accès au réseau (WebRequest, WebResponse, TcpClient, TcpListener, UdpClient, Sockets, etc.)
- **System.Security** : Gestion de la sécurité (Permissions, Cryptography, etc.)
- **System.Web.UI.WebControls** : Composants Web (BorderStyle, DataGrid, HyperLink, ListBox, Panel, RadioButton, Table, etc.)
- **System.Windows.Forms** : Composants graphiques classiques (Button, RichEdit, CheckBox, Form, ListBox, MainMenu, MonthCalendar, ToolBarTreeView, FileDialog, etc.)



Partie II

Le langage C#

Introduction

- C# est un langage à objets récent disponible officiellement depuis février 2002
- Evolution:
 - C# 1.0 : très proche de Java, mais reste syntaxiquement semblable au langages C et C++
 - C# 2.0 : apporte la notion des classes génériques, des classes partielles, des méthodes anonymes, etc.
 - C# 3.0 : intègre Linq, l'inférence des types locaux, les types anonymes, les initialiseurs d'objets, etc.
 - C# 4.0 : apporte les fonctionnalités de la programmation dynamique et de la coévolution des langages
 - C# 12 : la dernière version stable (2024)

Introduction

- C# est une amélioration de C++ et ressemble à Java en divers points
- C# fonctionne uniquement sur la plateforme .NET
- C# ne contient pas ses propres bibliothèques de classes, il utilise les classes du .NET Framework
- Le compilateur **csc.exe** de C# peut générer divers formats de sortie:
 - *Un module* : **csc** /**target:module** prog.cs ⇒ **prog.netmodule**
 - *Une DLL* : **csc** /**target:library** prog.cs ⇒ **prog.dll**
 - *Un exécutable console* : **csc** /**target:exe** prog.cs ⇒ **prog.exe**
 - *Un exécutable Windows* : **csc** /**target:winexe** prog.cs ⇒ **prog.exe**

Caractéristiques

- Syntaxe proche de C++ et Java
- La classe contient sa définition et son implémentation, dans un même fichier d'extension «.cs»
- Utilisation des *espaces de noms* pour le groupement de plusieurs classes
- Tout est classe même les types de base int , float, etc.
- Toutes les classes héritent de la classe racine **object** (`System.Object`)
- Seul l'opérateur d'accès aux membres `.` est utilisé
- Le compilateur C# peut générer un fichier de documentation XML à partir de commentaires spéciaux inclus dans le programme
- Généralisation de la gestion des exceptions
- L'héritage est simple mais une classe peut implémenter plusieurs interfaces
- Support natif des threads

Structure générale d'une application C#

- Syntaxe

```
//espaces de noms utilisés
using espace1;
using espace2.espace3;
using espace4;
//Déclaration d'un espace de noms
namespace NomEspace{
    class Class1{
        ...
    }
    ...
    class ClassPrincipale{
        ...
        static void Main(String[] args){
            ...
        }
    }
}
```

Structure générale d'une application C#

- Une application C# est composé d'une ou plusieurs classes
- Ces classes peuvent être réparties sur un ou plusieurs fichiers sources d'extension **.cs**
- Une seule parmi ces classes doit contenir la méthode principale **Main**
- Cette classe n'a pas forcément le même nom que le fichier source
- On peut mettre les classes dans un ou plusieurs espaces de noms
- Un espace de noms peut s'étaler sur un ou plusieurs fichiers sources
- On peut référencer d'autres espaces de noms en utilisant **using**
- Les **using** doivent précéder toutes les classes du fichier ou de l'espace de noms

Structure générale d'une application C#

- La méthode **Main** représente l'entrée à l'exécution de l'application
 - doit être qualifiée **static**
 - peut être de type **int** ou **void**
 - peut avoir ou non des arguments
- Les identificateurs en C# acceptent les lettres accentuées et les caractères '\$' , '_' et 'μ'
- En général, les espaces de noms, les classes et les méthodes commencent par une majuscule (C# respecte la casse)
- **using** permet l'utilisation non qualifiée des classes qui appartiennent à l'espace de noms (mais pas les classes des espaces de noms internes)

Structure générale d'une application C#

- Exemple : prog.cs

```
//espaces de noms utilisés
using System; //contient la classe Console
//Déclaration d'un espace de noms
namespace MonEspace{
    class Test{

        static void Main(String[] args){
            Console.WriteLine("Bonjour.");
        }
    }
}
```

Compilation sur la ligne de commande: > **csc** **prog.cs**

⇒Résultat : **prog.exe**

Les commentaires

- Les mêmes qu'en C++
- Nouveau commentaire :

///*ligne de texte*

Ce type de commentaires contient des balises XML qui permettront de générer un fichier de documentation XML lors de la compilation par l'option **/doc**

Exemple :

```
// prog.cs
using System;
///<remarks>
///description générale sur une classe</remarks>
public class Test {
    ///<summary>
    ///déscription d'un membre </summary>
```

...

Compilation : **csc** **prog.cs** **/doc:document.xml**

Les E/S standards : classe System.Console

- Méthodes d'écriture sur l'écran :

-Write(**expression**); // affiche une expression de tout type qui
// peut être converti en chaîne de caractères

Exemple : System.Console.Write("x= "+125);

 ou : System.Console.Out.Write(" x= " +125);

-WriteLine(**expression**); // affiche et revient à la ligne

- Méthodes de lecture à partir du clavier :

-Read(); // Lit un caractère et renvoie son code

Exemple : int i = System.Console.Read();

 ou : int i = System.Console.In.Read();

-ReadLine(); // Lit une chaîne jusqu'au retour à la ligne

Exemple : string s = System.Console.ReadLine();

-ReadKey(); // Lit un caractère à la volée

N.B: Pas de fonctions de lecture d'entiers ou réels (utilisation de Parse)

Opérateurs de C#

- Les opérateurs sont pratiquement les mêmes qu'en C++ (arithmétiques, logiques, binaires, etc)
- Les opérateurs logiques utilisent le type booléen (`true` ou `false`) au lieu de `int`
- **Plusieurs nouveaux Opérateurs**
 - `exp is nomType` : renvoie `true` si l'expression `exp` est de type `nomType`
 - `exp as nomType` : renvoie un objet représentant la conversion de l'expression `exp` en `nomType` ou `null` en cas d'échec
 - `typeof(exp)` : renvoie un objet de la classe `System.Type` permettant de décrire le type de l'expression `exp`
 - `operandeG ?? operandeD` : l'opérateur de fusion null `??` retourne l'opérande gauche si elle n'est pas nulle, sinon il retourne l'opérande droite

Structures de contrôle

- Les structures de contrôle **if-else**, **switch**, **for**, **while** et **do-while** sont les mêmes qu'en C++
- Les structures de contrôle utilisent le type **bool** (**true** ou **false**) au lieu de **int**
- **Switch** accepte les types entiers, énumérés et aussi le type **string**
- Nouvelle structure de contrôle **foreach** pour parcourir les tableaux et les collections

```
foreach(type variable in collection) {  
    instructions;  
}
```

variable prendra successivement tous les objets de **collection**
collection est une collection d'objets (exemple un tableau, une liste, etc.)
type est le type des objets de la collection

Exemple : **string[] T = { "Un", "Deux", "Trois" } ;**
 foreach (string ch in T)
 Console.WriteLine(ch) ;

La gestion des exceptions

- Traiter les erreurs exceptionnelles d'exécution telles que des données incorrectes, une base de données inaccessible, un fichier introuvable, etc.
- Plusieurs types d'exception dont la classe de base des exceptions est la classe **Exception**

- **Syntaxe:**

```
try {  
    //génération de l'exception  
}  
catch (TypeException1    e1) {  
    //traiter l'exception  e1  
}  
...  
catch (TypeExceptionk    ek) {  
    //traiter l'exception  ek  
}  
finally{    //optionnel  
    //code exécuté qu'il y a exception ou non  
}
```

La gestion des exceptions

Exemple : exceptions lors de la saisie de données incorrectes

```
try{
    int a = int.Parse(Console.ReadLine());
    int b = int.Parse(Console.ReadLine());
    int c = a/b;
    Console.WriteLine("resultat:" + c);
}
catch (OverflowException e){
    Console.WriteLine("erreur de saisie:"+e.Message);
}
catch (DivideByZeroException e){
    Console.WriteLine("erreur de division:"+e.Message);
}
finally{
    Console.WriteLine(' 'Fin du programme");
}
```

La gestion des exceptions

Remarques

- On peut utiliser **catch** sans paramètres si on n'a pas besoin de connaître des informations sur l'exception :

```
catch{  
    ...  
}
```

- On peut générer explicitement une exception à l'aide de l'instruction :

```
throw new RuntimeException("message") ;
```

- On peut relancer une exception après sa capture dans une clause catch à l'aide de l'instruction

```
throw ;
```

Types en C#

Tous les types de C# héritent de la classe racine **object**

Il existe deux catégories de types en C#:

• **Type valeur** : une variable de ce type est un objet destinée à contenir directement une valeur et comprend :

- Les types de base : **int** , **float**, **char**, etc.
- Le type **enum**
- Le type **struct**

```
TypeName obj;  
obj=valeur;
```

• **Type référence** : une variable de type référence est une sorte de pointeur destiné à contenir la référence vers un objet (l'adresse) ou **null**, et comprend :

- Les tableaux
- Les chaînes de caractères (**string**)
- Les classes et les interfaces (**class**, **interface**)
- Les délégués (**delegate**)

```
TypeName refObj;  
refObj=new TypeName();
```

Types valeur de C#

Types de base

Type	Désignation (taille)
char	caractère en Unicode '\uXXXX ' (16 bits)
int, uint	entier signé, non signé (32 bits)
long, ulong	entier signé, non signé (64 bits)
sbyte, byte	entier signé, non signé (8 bits)
short, ushort	entier signé, non signé (16 bits)
float	réel simple précision (32 bits)
double	réel double précision (64 bits)
decimal	réel large précision (128 bits)
bool	booléen (1 bit)

Types valeur de C#

Classes équivalentes dans .NET

Type valeur de C#	Classe équivalente dans .NET
byte	System.Byte
sbyte	System.SByte
char	System.Char
short	System.Int16
ushort	System.UInt16
int	System.Int32
uint	System.UInt32
long	System.Int64
ulong	System.UInt64
float	System.Single
double	System.Double
decimal	System.Decimal
bool	System.Boolean

Types valeur de C#

TP 1

Développer une mini calculatrice (+,-,*,/, puissance ^, racine r) avec gestion des erreurs.

Types valeur de C#

Type enum

- Un type **enum** (énumération) est un type valeur qui permet de définir une classe (ensemble) de constantes
- Ces constantes sont des valeurs entières à partir de 0
- Un type **enum** est équivalent à la classe **System.Enum**
- **Déclaration :** `enum nomType {Nom1, Nom2, ...}`

Exemple :

```
enum Etat{Fermé, Ouvert} //Ferme←0 et Ouvert←1
Etat      etat0 = Etat.Fermé;
int        val= (int)etat0 ;      //val ← 0
Console.WriteLine( "état="+etat0); // état=Fermé
```

Remarque : Il est possible d'attribuer des valeurs personnalisées à une ou à plusieurs constantes de l'énumération

Exemple :

```
enum Climat { froid=-10, chaud=37}
```

Types valeur de C#

Type struct

- Un type **struct** définit une classe dont les instances sont des objets et non des références (l'opération new n'est pas nécessaire)
- On ne peut pas hériter à partir d'une classe **struct**
- Les classes des types de base dans .NET comme **int** (**System.Int32**) **char**, **float**, **DateTime**, etc. sont de type **struct**

- **Déclaration**

```
struct          nomType {  
    // attributs  
    // méthodes et constructeurs  
}
```

Types valeur de C#

Type struct

•Exemple :

```
struct personne{
    public string nom;
    public int age;
}
public class test {
    public static void Main() {
        personne    p1;
        p1.nom = "Ali";
        p1.age = 10;
        personne    p2 = p1;    //recopie du contenu
    }
}
```

Types référence de C#

La classe object

- **object** est la classe de base de toutes les classes en C#
- **object** est l'équivalent de la classe **System.Object**
- Les méthodes de la classe **object** :

```
public object()           //constructeur
public string ToString()  //conversion vers chaîne
public bool Equals(object) //teste l'égalité
public System.Type GetType() //renvoie le type de l'objet
...
```

Types référence de C#

Les tableaux

- Les tableaux dérivent de la classe abstraite **System.Array**
- Tout débordement du tableau génère une exception de type **IndexOutOfRangeException**
- Tableau unidimensionnel

```
Type[] Tab = new Type[dimension];
```

↳ Initialisation lors de la déclaration :

```
Type[] Tab = new Type[] {val0, val1, val2, ...};
```

ou:

```
Type[] Tab = {val0, val1, val2, ...};
```

ou:

```
Type[] Tab;
```

```
Tab = new Type[] {val0, val1, val2, ...};
```

↳ La propriété **Length** contient le nombre d'éléments

```
int l = Tab.Length;
```

Types référence de C#

Les tableaux

•Copie de tableaux

↳ La méthode **Clone** : renvoie une référence sur un nouveau objet copie d'un tableau **tab1**

```
Type [] tab2 = (Type[]) tab1.Clone();
```

↳ La méthode statique **Copy** de **Array** : permet de copier du tableau **tab1** vers **tab2** un nombre d'éléments **nbelements**

```
Array.Copy (tab1, tab2, nbElements);
```

↳ Syntaxe complète: **Array.Copy (tab1, indice1, tab2, indice2, nbElements);**

↳ La méthode **CopyTo** : permet de copier dans un tableau **tab2** tous les éléments d'un tableau **tab1** en les stockant à partir de l'indice **i** de **tab2**

```
tab1.CopyTo (tab2, i);
```

•Max et Min de tableaux

Les méthodes **tab.Max()** et **tab.Min()** retrouvent le max et le min du tableau **tab**

•Tri de tableaux

La méthode statique **Array.Sort (tab)** trie un tableau **tab** dans l'ordre croissant par défaut

•Inversion de l'ordre d'un tableau

La méthode statique **Array.Reverse (tab)** inverse l'ordre des éléments de tout le tableau **tab**

•Redimensionnement d'un tableau

La méthode statique **Array.Resize(ref tab, newDim)** redimensionne le tableau **tab** avec la nouvelle dimension **newDim** en préservant le contenu

Types référence de C#

Les tableaux

• Plage d'éléments d'un tableau

➡ On peut accéder à une plage d'éléments d'un tableau:

- Tout le tableau:

```
tab1=tab2[...];
```

- Une plage entre deux indices du tableau:

```
tab1=tab2[indice1..indice2]; (indice1<=plage<indice2)
```

- A partir d'un indice jusqu'à la fin du tableau:

```
tab1=tab2[indice1..];
```

- A partir du début jusqu'à un indice donné:

```
tab1=tab2[..indice];
```

• Position à partir de la fin d'un tableau

- Un élément à la position pos à partir de la fin du tableau:

```
element=tab2[^pos]; (pos>=1)
```

- Plage d'éléments entre deux positions par rapport à la fin du tableau:

```
tab1=tab2[^pos2..^pos1];
```


Types référence de C#

Les tableaux

- Tableau à plusieurs dimensions

Chaque ligne contient le même nombre de cases

```
Type[, , ...] Tab=new Type [dim1,dim2,dim3,...];
```

Exemple : Tableau à deux dimensions

```
int[,] T1= new int[7,5]; //7 lignes et 5 colonnes
int[,] T2= new int[3,2] {{0,1},{3,4},{6,7}};
int[,] T3= new int[,] {{0,1},{3,4},{6,7}};
int[,] T4= {{0,1}, {3,4}, {6,7}};
int n=T1.Length; //n←35=7*5:nombre total de cases
int i=T2[1,0]; // i←3:2ème ligne et 1ère colonne
int L1= T3.GetLength(1); /*L1←2 : nombre de
    colonnes de la 2ème ligne */
```

Types référence de C#

Les tableaux

•Tableau de tableaux (tableau en escalier)

- Chaque ligne contient son propre nombre de cases
- On ne doit spécifier que la 1ère dimension

```
Type[][]...[] Tab=new Type[dim1][]...[];
```

Exemple : Tableau à deux dimensions

```
int[][] T1= new int[2][ ];  
int[][] T2= { new int[]{2,3,4} ,  
              new int[]{5,6,7,8,9}  
            };  
T1[0]=new int[3];  
T1[0][0]=15;  
T1[1]=new int[5];
```

Types référence de C#

Les chaînes de caractères

•Le type `string`

- Le type `string` est un alias de la classe `System.String`
- Une chaîne de type `string` ne peut pas être modifiée

⇒ Déclaration

```
string chaine; //chaîne non initialisée!
```

⇒ Déclaration avec initialisation

```
string chaine = new string("message");
```

```
string chaine = "message";
```

```
string chaine = new string(c,n); //n caractères c répétés  
string chaine = new string(t); //t un tableau de caractères
```

⇒ Affectation par l'opérateur = :

```
chaine = "message";
```

⇒ Test d'égalité (inégalité) des chaînes par l'opérateur == (et !=) :

```
if (chaine1 == chaine2) ...
```

⇒ Comparaison des chaînes par la fonction `CompareTo` de type int:

```
if (chaine1.CompareTo(chaine2) < 0) ...
```

Types référence de C#

Les chaînes de caractères

⇒ Longueur d'une chaîne par la propriété `Length` :

```
int l = chaine.Length;
```

⇒ Accès à un caractère de la chaîne par l'opérateur `[]` :

La chaîne est vue comme un tableau de char

Exemple :

```
string ch = "abcdef";
```

```
char c = ch[4] ; //c ← 'e'
```

```
ch[5] = 'x' ; //erreur: modification interdite!
```

⇒ Concaténation de deux chaînes par l'opérateur `+` et `+=`:

```
string str1, str2, str3;
```

```
str3 = str1 + str2;
```

```
str3 += "Test";
```

Types référence de C#

Les chaînes de caractères

⇒ Insertion d'une chaîne :

```
string  str1, str2, str3;  
str2 = str1.Insert(pos, str3);
```

⇒ Recherche de la position de la 1ère occurrence d'une sous chaîne :

```
int  pos  = chaine.IndexOf ( sousChaine );
```

⇒ Conversion d'un nombre en chaîne :

```
""+nombre      ou  variable.ToString()  ou  
variable.ToString("format")
```

⇒ Conversion d'une chaîne en nombre :

Tous les types de base possèdent la méthode statique **Parse**

```
int    i = int.Parse(chaine);  
ou:    i=Int32.Parse(chaine);  
float  x = float.Parse(chaine);
```

Types référence de C#

Les chaînes de caractères

⇒ Interpolation de chaînes

- Le symbole **\$** permet d'injecter les valeurs des variables dans une chaîne de caractères grâce aux symboles **{ }**

Exemple:

```
string    str1;  
int v=5;  
float x=3.1445f;  
str1=$" v a pour valeur {v} et l'arrondie de x est {x:N2}";  
//x sera affiché au format Numérique avec 2 décimales
```

- Le symbole **@** permet d'inhiber les caractères d'échappement (****) dans une chaîne

Exemple:

```
string    str1= @" le path est c:\Windows\System32",  
          str2= @"Bonjour  
                tout le monde"; //saut de ligne= \n
```

TP 2

On veut proposer quelques services pour gérer un stock de produits alimentaires en utilisant un tableaux de type string.

- Chaque produit est stocké dans une chaine de caractères sous la forme :
"nom, catégorie, prix, dateProduction, dateExpiration"
- La catégorie est un type énuméré : Fromage, Lait, Legume ou Fruit

Ecrire des fonctions pour:

- Saisir un tableau de produits avec gestion des des erreurs de saisie
- Faire plusieurs type d'affichage des produits: afficher par ordre alphabétique, par catégorie, par prix décroissant, par date d'expiration croissante.
- Chercher et retourner les produits par catégorie
- Chercher et retourner les produits qui vont expirer dans 10 jours
- Calculer la somme prévisionnelle des pertes
- Trier les produits par ordre d'expiration la plus proche
- Retirer les produits qui vont expirer aujourd'hui

Types référence de C#

Les classes

- Une classe en C# est accessible par défaut partout dans l'assemblée où elle a été définie, elle est implicitement déclarée `internal class NomClasse{}`
- Toutes les classes héritent automatiquement de la classe `object`

```
class NomClasse{  
    //attributs ou champs  
    ...  
    //propriétés  
    ...  
    //méthodes  
    ...  
}
```

- L'instanciation de la classe permet de créer un objet en mémoire qui sera référencé par une variable appelée référence (pointeur):

```
NomClasse    ref; //ref est égale à null ici  
ref = new NomClasse(...); //appel au constructeur pour l'initialisation  
                        //de l'objet crée
```


Types référence de C#

Les classes

- Les attributs ou champs ou données de la classe

```
class NomClasse {  
    type1 nomAttribut1;  
    //attributs initialisés directement  
    type2      nomAttribut2=new type2(listValeurs);  
    //attributs constants → implicitement statiques  
    const type3 nomAttribut3 = new type3(listValeurs);  
    /*attributs en lecture seule  
    ⇒ peuvent être affectés une seule fois */  
    readonly type4 nomAttribut4=new type4(listValeurs);  
    readonly type5 nomAttribut5;  
    public NomClasse(...) {      //constructeur  
        nomAttribut1 = new type1(listValeurs);  
        nomAttribut5 = new type5(listvaleurs);  
  
        ...  
    }  
}
```

Types référence de C#

Les classes

- Exemple

```
class Employe{
    int matricule;
    String nom;
    int heureTravail;
    String pays="Maroc";
    const double taxeIGR = 17.00;
    readonly double salaire;
    public Employe(){
        ...
        salaire= calculSalaire(heuresTravail);
    }
    ...
}
```

Types référence de C#

Les classes

•Visibilité d'une classe

- Une classe C# est **internal** par défaut càd accessible dans tout le projet (assembly) où elle a été définie

class NomClasse{...} équivaut à : **internal class NomClasse{...}**

- Une classe qualifiée **public** est accessible même à l'extérieur de son assembly
- Une classe interne est définie à l'intérieur d'une autre classe conteneur
 - Elle est **private** par défaut donc inaccessible de l'extérieur de la classe conteneur
 - On peut modifier ce type d'accès par :
 - ➡ **public** : classe visible à partir d'une autre assembly où la classe conteneur est visible aussi
 - ➡ **protected** : classe visible uniquement pour les classes qui héritent de la classe conteneur
 - ➡ **internal** : classe visible aussi dans tous les namespace de l'assembly où la classe conteneur est visible aussi

Types référence de C#

Les classes

- **Visibilité des membres de la classe**

- Les membres de la classe sont **private** par défaut et donc accessibles uniquement à l'intérieur de la classe

[private] type NomMembre;

- On peut utiliser les modificateurs d'accès suivants :

↳ **protected** : les membres sont visibles pour les classes dérivées de cette classe

↳ **internal** : ils sont visibles pour toutes les classes incluses dans la même assembly

↳ **public** : ils sont visibles pour les classes d'une autre assembly où la classe est visible

Types référence de C#

Les classes

•Les constructeurs

- Même syntaxe et sémantique qu'en C++ (mais pas de constructeur de copie)

```
NomClasse(list_parametres)
{
    ...
}
```

- Un constructeur est exécuté lors de l'instanciation de la classe :

```
NomClasse refobjet= new NomClasse(list_valeurs) ;
```

- Un constructeur peut appeler un autre constructeur de la même classe en utilisant **this** :

```
NomClasse(list_parametres) : this(list_valeurs) {
    ...
}
```

Remarque: Dans le code des fonctions le mot clé **this** peut être utilisé pour accéder aux membres de la classe (surtout lorsque les paramètres des fonctions ont le même nom que les attributs): **this.nomMembre**

Types référence de C#

Les classes

- **Le destructeur (très rarement utilisé!)**

- Même syntaxe et sémantique qu'en C++
- Appelé automatiquement lorsque le Garbage Collector détruit une instance de la classe
- Le destructeur appelle automatiquement les destructeurs de toutes les classes de base de manière ascendante

```
class NomClasse {  
    ~NomClasse() {  
        ...  
    }  
}
```

Types référence de C#

Les classes

•Les propriétés

- Une propriété permet de faciliter l'accès à des attributs privés de la classe
- Une propriété permet d'éviter l'utilisation fastidieuse de **getters** et **setters**
- Une propriété permet d'extraire ou de spécifier la valeur d'un attribut, elle encapsule deux méthodes **get** et **set** (ou juste l'une des deux)

```
public type nomPropriete {  
    get{ ...  
        return nomAttribut;  
    }  
    set{ ...  
        nomAttribut = value;  
    }  
}
```

value représente automatiquement la valeur qui sera affectée à l'attribut

Remarque : On doit utiliser évidemment un appel à une fonction qui réalise l'affectation si le type de l'attribut ne supporte pas l'opérateur **=**

Types référence de C#

Les classes

- **Exemple**

```
class PersonneAdulte{
    // attributs
    private string nom;
    private int age;
    // propriétés
    public string Nom{
        get{ return nom; }
        set{ nom = value; }
    }
    public int Age{
        get{ return age; }
        set{ if(value>=18)    age=value;
              else age = 18;}
    }
    public static void main(){
        PersonneAdulte  p = new PersonneAdulte ();
        p.Nom = "Ali";      // nom ← "Ali";
        p.Age = 30;         // age ← 30
        Console.WriteLine(p.Nom); // appel au get
    }
}
```


Types référence de C#

Initialisation d'objet

- C'est une simplicité du langage permettant d'initialiser un objet en utilisant ses propriétés publiques sans créer de constructeur particulier

- **Syntaxe**

```
NomClasse variable= new NomClasse {Propriete1=valeur1,  
                                     Propriete2=valeur2,... } ;
```

- Les propriétés peuvent être données dans n'importe quel ordre
- Si aucune propriété n'est mentionnée alors l'initialisation sera effectuée par le constructeur par défaut

```
NomClasse variable= new NomClasse {} ;
```

Types référence de C#

Inférence de type

- C'est une simplicité du langage qui peut être utilisée lors de la déclaration d'une variable avec initialisation, on peut alors utiliser le mot clé **var** à la place du type.

- **Syntaxe**

```
var variable = new Type();  
var variable = expression;
```

Remarque: la variable ne peut pas être initialisée à la valeur **null**

- **Exemple**

```
var i = 1;  
var s = "Bonjour";  
var t = new[] { 0, 1, 2 };  
var list = new List<Produit>();  
foreach(var item in list){ ... }
```

Types référence de C#

Initialisation d'objet

- Exemple

```
class Produit
{
    int code;
    String nom;
    float prix;
    public int Code{ get;set;}
    public string Nom{ get;set;}
    ...
}

Produit p1=new Produit{Code=100, Nom="TV", Prix=5000};
Produit p2=new Produit{Nom="PC",Code=200};
Produit p3=new Produit{}; //constructeur par défaut
```

Types référence de C#

Agrégation de classes

- L'agrégation est un concept très utilisé en programmation orientée objet
- Elle permet de représenter **une relation** entre un objet appelé «**agrégat**» ou «**ensemble**» et d'autres objets de type différent appelés «**éléments**»
- On parle d'une relation de type : «**ensemble–éléments**»
- Cette relation modélise également des relations telles que «contient», «possède», «correspond à», «constitué de», etc.
- Les objets « ensemble » et « élément » peuvent exister indépendamment l'un de l'autre
- La destruction de l'objet « ensemble » n'entraîne pas forcément la destruction automatique de l'objet « élément »

Types référence de C#

Agrégation de classes

- Syntaxe

```
class ClasseEnsemble
{
    ClasseElement1 element1;

    ClasseElement2 element2;
}
```

```
class ClasseEnsemble
{
    ClasseElement1[] elements;
}
```

Types référence de C#

Agrégation de classes

•Exemple

```
class Stock
{
    Produit[] produits; //un stock contient plusieurs produits
    ...
}
class Produit
{
    int code;
    string designation;
    float prix;
    Fournisseur fournisseur; // un produit correspond à
                             //un fournisseur
    ...
}
```

```
class Fournisseur
{
    String nomSociété;
    String adresse;
    ...
}
```

Types référence de C#

Composition de classes

- La composition est une forme **forte d'agrégation**
- Elle représente une relation «**est composé de**» entre un objet appelé «**composite**» et des objets de type différent appelés «**composants**»
- La dépendance est forte entre «**composite**» et «**composants**» c-à-d que:
 - Les composants ne peuvent pas exister indépendamment du composite
 - Le composite gère entièrement le cycle de vie de ses composants
 - Si l'objet composite est détruit, les composants sont également détruits
- On peut implémenter ce concept en C# en utilisant les **champs privés** (sans getter ni setter) ou carrément des **classes internes**

Types référence de C#

Composition de classes

•Syntaxe

```
//avec classe interne:  
class ClasseComposite  
{  
    ClasseComposant composant;  
    ...  
  
    //classe interne privée:  
    class ClasseComposant  
    {  
        ...  
    }  
}
```

```
//avec champ privé:  
class ClasseComposite  
{  
    ClasseComposant composant;  
    ...  
    //ni getter ni setter  
    //pour composant  
}  
  
class ClasseComposant  
{  
    ...  
}
```


Types référence de C#

Composition de classes

- **Exemple 1 (avec champ privé):** un hôtel est composé de plusieurs chambres et une chambre n'existe pas sans hôtel

```
class Hotel
{
    String nom;
    String adresse;
    Category categorie;
    Chambre[] chambres; //champ privé pas de getter ni setter
    ...
    public void AjouterChambre(int numero, int numEtage, TypeChambre type)
    {
        Chambre chambre= new Chambre(...); //créer la chambre ici
    }
}
class Chambre
{
    int numero;
    int numEtage;
    TypeChambre type;
    ...
}
```

Types référence de C#

Composition de classes

- **Exemple 2 (avec classe interne):** une liste chaînée est composée de plusieurs cellules et une cellule ne doit pas être créée à l'extérieur de sa liste

```
class    ListeChaineFournisseurs
{
    Cellule tete;
    public void AjouterFournisseur(Fournisseur fournisseur) {
    ...
    }
    ...
    //classe interne
    class Cellule
    {
        Fournisseur val;
        Cellule suivant;

        ...
    }
}
```

Exercice

On souhaite développer une application permettant de gérer une compétition internationale de football.

La compétition est caractérisée par le nom de la compétition, l'année, le pays organisateur et l'ensemble des équipes participantes.

Une équipe est caractérisée par le nom de l'équipe, le pays, l'entraîneur, un ensemble de joueurs appartenant à l'équipe.

L'entraîneur est caractérisé par son nom et prénom et sa nationalité.

Chaque joueur est identifié par son nom et prénom, son numéro et son poste sur le terrain (ailier, attaquant, etc.).

Chaque joueur dispose de statistiques personnelles telles que le nombre de matchs joués, le nombre de buts marqués et le nombre de passes décisives.

A chaque tour de la qualification on doit garder: le nom du tour (tour 16eme, tour 8eme, quart de final, demi finale, finale), les équipes participantes et les équipes qualifiées.

Chaque équipe dispose à chaque tour de statistiques sur le nombre de matchs joués, le nombre de matchs gagnés, le nombre de matchs perdus, le nombre de points, le nombre de buts marqués, le nombre de buts reçus.

Les équipes avec le plus de points sont qualifiées au tour suivant et en cas d'égalité, utiliser la différence de buts

Travail demandé

- Identifier les classes nécessaires permettant de résoudre le problème.
- Pour chaque classe : ajouter des constructeurs, définir les propriétés appropriées et redéfinir la méthode ToString().
- Ajouter des méthodes (fonctions) adéquates aux classes afin de permettre le test suivant dans le programme principal (Main) :
 - Créer une compétition internationale de football avec ses équipes et leurs joueurs.
 - Mettre en place une boucle sur les tours de qualification et y appliquer les traitements suivants :
 - Créer un tour de qualification avec les équipes participantes ;
 - Pour chaque équipe participante à un tour donné, saisir ses statistiques
 - Comparer les équipes d'un même tour afin de déterminer les équipes qualifiées
 - Afficher à l'écran la liste des équipes qualifiées pour le tour suivant
 - Si le tour est « Finale », afficher l'équipe gagnante avec ses statistiques et le meilleur joueur de la compétition.

Types référence de C#

Les classes génériques

- Même concept que les classes « template » de C++
- Ce sont des classes paramétrées par une suite de types génériques
- **Syntaxe de classe générique**

```
class    NomClasse<T1,T2,.....,Tk>
{
    ....
}
```

Où **T1**, **T2** à **Tk** sont des identifiants de type génériques

- Les génériques évitent d'utiliser un casting et permettent donc d'avoir un code plus sécurisé (code-safe) : un casting n'est analysé que lors de l'exécution ce qui peut générer une exception.
- Les génériques du C# peuvent être une Classe, Interface , Méthode ou Délégué paramétrés par un ou plusieurs types génériques

Types référence de C#

Les classes génériques

- **Exemple** : liste chaînée générique

```
class ListeChaine<T>{  
    public void Ajouter(T element) {  
        ...  
    }  
    public void Supprimer(T element) {  
        ...  
    }  
    ...  
}
```

Instanciation:

```
ListeChaine<Fournisseur> list1=new ListeChaine<Fournisseur>();  
ListeChaine<int> list12=new ListeChaine<int>();
```

Types référence de C#

Les collections et les collections génériques

- L'espace de noms `System.Collections` contient les classes représentant les structures de données les plus couramment utilisées :
 - `ArrayList` : une liste dynamique qui peut contenir des éléments de types différents
 - `Queue` : une file d'attente qui suit le principe FIFO (First In, First Out)
 - `Stack` : une pile suivant le principe LIFO (Last In, First Out)
 - `Hashtable` : une table de hachage permettant de stocker des paires clé-valeur
- L'espace de noms `System.Collections.Generic` contient plusieurs collections génériques fortement typée (type safe) et plus recommandées
 - `List<T>` : une liste générique similaire à `ArrayList`
 - `Stack<T>` : une pile générique (LIFO)
 - `Queue<T>` : une file d'attente générique (FIFO)
 - `Dictionary<TKey, TValue>` : une table de hachage générique

Types référence de C#

Les collections et les collections génériques

- Exemple: une liste de réels à l'aide de `List<>` générique

```
List<double> liste = new List<double>();

for (int i=0;i<10;i++)
    liste.Add(i*3.14);

liste.RemoveAt(3); //suppression par la position
Console.WriteLine(liste.Contains(6.28));
Console.WriteLine("pos="+liste.IndexOf(6.28));
foreach (var element in liste)
    Console.WriteLine(element);
for (int j = 0; j < liste.Count; j++ )
{
    Console.WriteLine(liste[j]); //l'indexeur
}
```


Types référence de C#

L'héritage

L'héritage est un concept fondamental en POO, il représente une relation « **est un** » (**is-a**) entre des classes, exemple: un **employé** **est un** **salarié** de l'entreprise

- Une classe ne peut hériter que d'une seule classe de base : **héritage simple**

```
class NomClass : ClassBase  
{...}
```

- Les constructeurs et les destructeurs ne sont pas hérités, chaque classe doit définir ses propres constructeurs et destructeur

- Une classe peut hériter d'une classe de base et implémenter plusieurs **interfaces**

```
class NomClass : ClassBase, Interface1, ..., InterfaceK  
{...}
```

- On ne peut pas hériter d'une classe qualifiée par **sealed** :

```
sealed class NomClass ...
```

Types référence de C#

L'héritage

- Un constructeur de la classe appelle toujours un constructeur de sa classe de base en utilisant le mot clé **base** :

```
NomClasse(list_parametres) : base(list_valeurs)
{...}
```

- Si on écrit uniquement :

```
NomClasse(list_parametres)
{...}
```

ceci équivaut à un appel au constructeur par défaut de la classe de base :

```
NomClasse(list_parametres) : base()
{...}
```

- Une classe dérivée peut masquer des membres hérités en déclarant de nouveaux membres ayant le même nom ou la même signature
- Le mot clé **base** peut alors être utilisé pour accéder aux membres de la classe de base

```
base.nomMethode(list_valeurs);
```

TP 3

Réaliser une application pour gérer les salariés d'une entreprise c'est-à-dire l'ajout, la recherche multicritères, la suppression et la modification des salariés. Un salarié est soit un employé, un chef de service ou un directeur.

Un salarié est caractérisé par son nom, prénom, statut (marié ou célibataire), sa date d'embauche et salaire.

Un employé est un salarié qui possède un matricule unique et une fonction au sein de l'entreprise. Il perçoit un salaire mensuel selon le nombre d'heures de travail et un taux horaire. Il a aussi une indemnité familiale s'il est marié et une indemnité d'ancienneté.

Nous devons aussi gérer les chefs de service qui sont des employés spéciaux dont la fonction est la supervision d'un service donné de l'entreprise (service comptabilité, service stock, etc.) et perçoivent pour cela une indemnité de chef de service en plus de leur salaire.

Les directeurs sont des hauts salariés qui possèdent un diplôme supérieur (ingénieur, master ou doctorat) et une expérience de plusieurs années. Ils perçoivent un salaire différent selon leur diplôme et possèdent plusieurs indemnités: indemnité familiale s'il est marié, une indemnité d'ancienneté, une indemnité de déplacement et une indemnité de logement.

Types référence de C#

Le polymorphisme

Une méthode est dite polymorphe si elle apparaît dans plusieurs classes avec un code différent, on dit qu'elle possède un comportement différent en fonction de la classe

- Une méthode est polymorphe si elle est qualifiée **virtual** dans la classe de base
- Dans les classes dérivées elle doit être qualifiée **override** et doit garder la même signature

```
class Base {  
    virtual type methode(list_params)  
    {...}  
}  
class Derive : Base {  
    override type methode(list_params)  
    {...} //traitement different  
}
```

- Si on ajoute le qualificatif **sealed** la méthode ne pourra plus être polymorphe pour les prochaines dérivations : **sealed override type methode(list_params) {...}**

Types référence de C#

Les méthodes et classes abstraites

❑ Classes abstraites

« Une classe abstraite représente un concept général qui regroupe un ensemble d'objets concrets. La classe abstraite ne peut pas être instanciée et sert de classe de base pour les classes concrètes de ces objets »

- **Exemple:** les **employés et les chefs de service** **sont des personnes** qui travaillent pour l'entreprise
 - **Personne** est une classe abstraite qui regroupe les caractéristiques communes des employés et des chefs de service
 - **Employe** et **ChefService** sont des classes concrètes instanciables
- **Syntaxe**

```
abstract class NomClasse {...}
```

❑ Méthodes abstraites

« Une méthode abstraite est une méthode dont on connaît la signature (l'objectif, le quoi), mais dont l'implémentation est absente (le comment), et qui sera définie dans les classes dérivées »

```
public abstract type nomMethode(parametres);
```

- Les méthodes abstraites sont **polymorphes**
- Les méthodes abstraites ne doivent pas être déclarées **private**
- Elles doivent être redéfinies dans les classes dérivées avec le qualificateur **override**
- Toute classe contenant une méthode abstraite devient une classe abstraite

Types référence de C#

Compatibilité entre classe de base et ses classes dérivées

- Une classe dérivée est compatible avec sa classe de base
- Donc, une référence sur une classe dérivée peut remplacer une référence de la classe de base, l'inverse est une erreur!

```
ClasseBase      v1;
```

```
ClasseDerive    v2;
```

```
v1=v2; //correct
```

Par contre:

```
v2=v1; //erreur! Nécessite une conversion de type
```

- De même si on a une fonction avec en paramètre une référence de la classe de base alors on peut faire l'appel avec une référence de la classe dérivée:

```
TypeFonction  NomFonction(ClasseBase p){...}
```

Lors de l'appel:

```
NomFonction(v2); //correct
```

Types référence de C#

Les interfaces

Une interface est un contrat fonctionnel qui spécifie une liste de services (fonctions) qui seront respectés et implémentés par d'autres classes

```
interface nomInterface
{
    type1 methode1(list_params);
    ...
    typek methodek(list_params);
}
```

- Une interface contient les signatures des méthodes pas leur implémentation
- Une interface ne contient pas d'attributs (sauf rarement des attributs constants)
- Une interface ne contient ni constructeur ni méthode statique, donc elle n'est pas instanciable
- Les méthodes d'une interface seront implémentés dans les classes qui héritent de l'interface
- Une interface peut hériter à partir d'autres interfaces

Partie 2

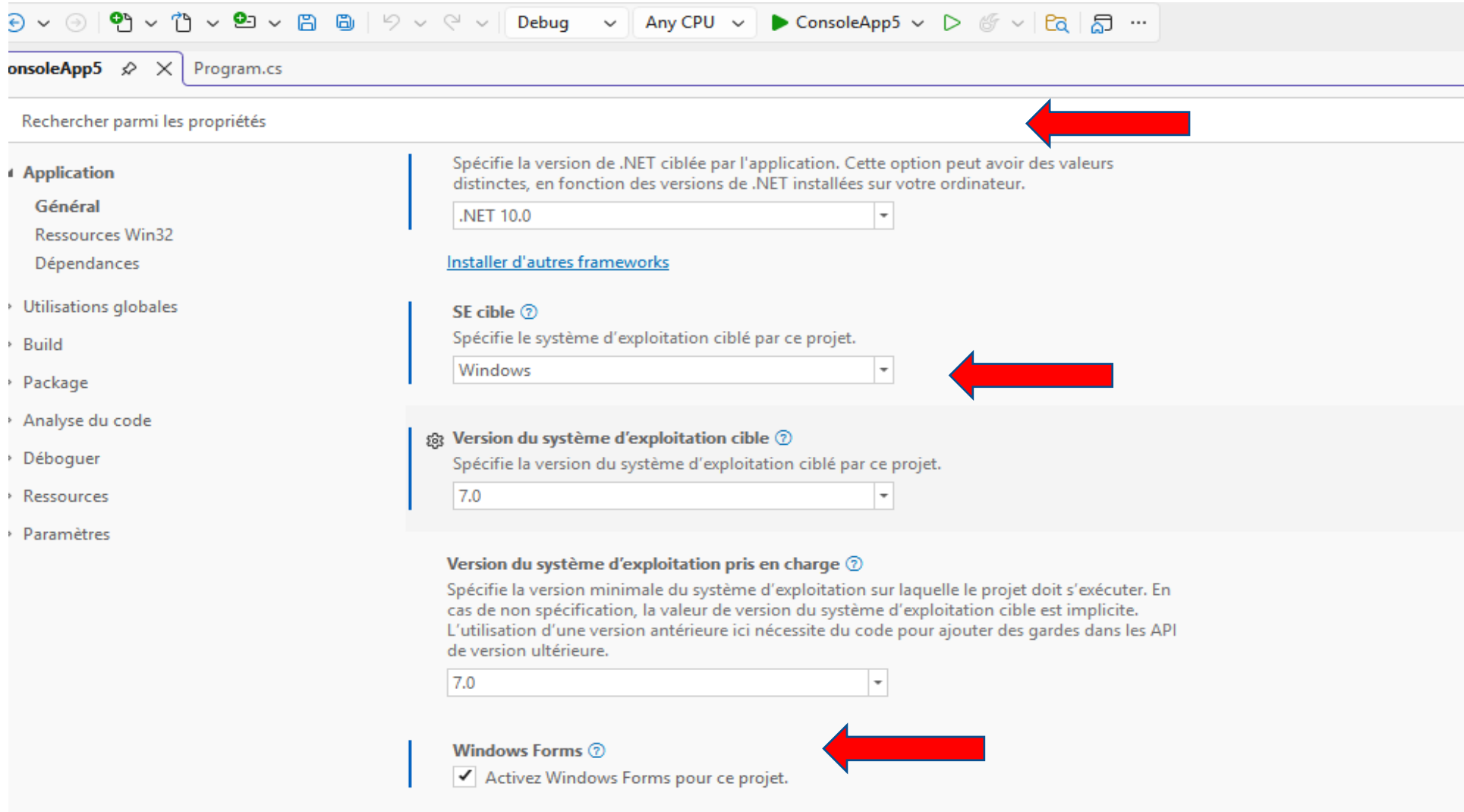
Interfaces graphiques en C#

Interfaces graphiques en C#

Les principales technologies pour les applications bureaux (**Desktop**) sont:

- ❑ **Windows Forms (WinForms)**: Technologie classique pour créer des applications Windows avec interface graphique. Elle est basée sur l'espace de noms `System.Windows.Forms`
- ❑ **WPF (Windows Presentation Foundation)**: Technologie plus moderne et puissante que Windows Forms. Elle utilise le langage XAML pour concevoir l'interface graphique
- ❑ **.NET MAUI (Multi-platform App UI)**: Technologie récente qui permet de créer des applications multiplateformes (Windows, Android, iOS, macOS) avec un seul code.

Modifier les propriétés suivantes du projet



Windows Forms → System.Windows.Forms

Les classes de cet espace de noms se répartissent en 3 catégories :

- ❑ La classe **Control** représente la classe de base des contrôles standards tels que : **Label**, **TextBox**, **Button**, **CheckBox**, **RadioButton**, **GroupBox**, **ListBox**, **ComboBox**, **PictureBox**, **MonthCalendar**, **StatusBar**, **NumericUpDown**,...
- En plus des contrôles conteneurs qui permettent d'organiser ces contrôles tels que : **Form** (fenêtre principale), **Panel**, **GroupBox**, **TabControl**, **TabPage**,...
- ❑ La classe **Component** définit la classe de base pour d'autres contrôles tels que : **MenuStrip**, **ToolStrip**, **ToolStripMenuItem**, **ContextMenuStrip**, **ToolTip**, **Timer**, ...
- ❑ La classe **CommonDialog** définit la classe de base pour les boîtes de dialogue communes: **OpenFileDialog**, **SaveFileDialog**, **FontDialog**, **PageSetupDialog**, **PrintPreviewDialog**, **PrintDialog**, **ColorDialog** et **FolderBrowserDialog**

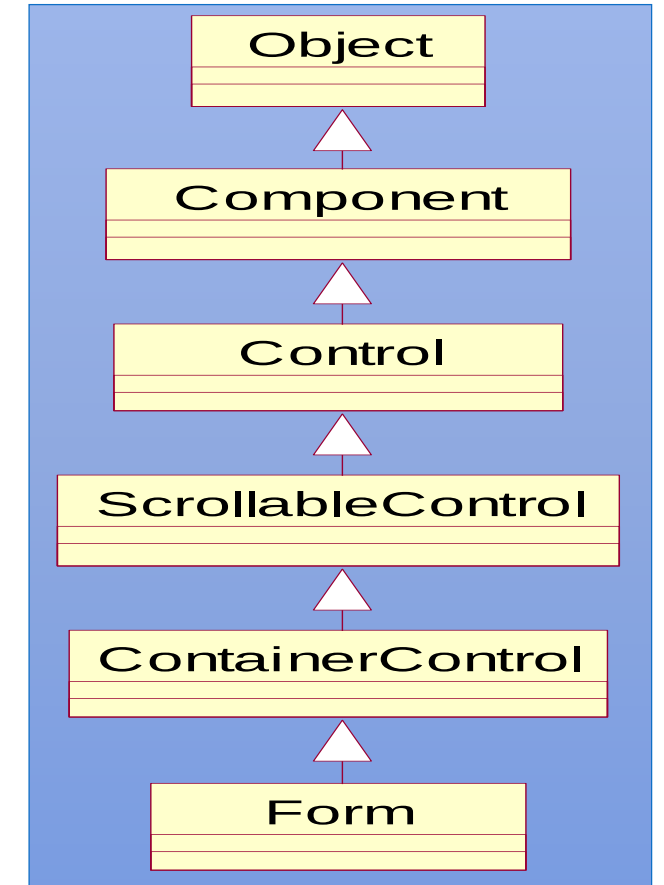
Windows Forms : `System.Windows.Forms`

La classe `Form`

❑ Une fenêtre est une classe qui dérive de la classe `Form`, elle peut être :

- Une boîte de dialogue
- Une fenêtre modale ou non modale
- Une fenêtre fille
- Une fenêtre parent d'une fenêtre fille

❑ La classe `Form` hérite indirectement de la classe `Control`



Windows Forms : System.Windows.Forms

La classe Form

❑ Quelques propriétés :

- **Size** : permet de spécifier ou retrouver la taille de la fenêtre nommée **f**
`f.Size = new Size(300, 200);`
- **Visible** : permet de contrôler la visibilité de la fenêtre
`f.Visible = false;`
- **Text** : permet de spécifier le titre de la fenêtre
`f.Text = "Ma 1ère Form";`
- **Controls** : permet d'ajouter ou supprimer des contrôles dans la fenêtre
`f.Controls.Add(button1);`
- **BackColor** : permet de spécifier la couleur de fond
`f.BackColor = Color.Blue;`
- **ForeColor** : permet de spécifier la couleur du texte
`f.ForeColor = Color.White;`
- **Font** : gère la police du texte affiché
`f.Font = new Font("Times New Roman", 12);`
- **BackgroundImage** : Obtient ou définit l'image (type **Image**) en arrière plan
- **MainMenuStrip** : indique ou retrouve le menu affiché dans la fenêtre
`f.MainMenuStrip = new MenuStrip();`

Windows Forms : `System.Windows.Forms`

La classe Form

❑ Quelques méthodes :

- `Form()` : l'unique constructeur
- `Show()` / `Hide()` : affiche / cache la fenêtre
- `ShowDialog()` : permet d'afficher la fenêtre comme une boîte de dialogue
 - La propriété `DialogResult` représente dans ce cas le résultat de la boîte de dialogue
- `Close()` : ferme la fenêtre

❑ Quelques événements :

- `Load` : se produit lors du chargement de la fenêtre
- `FormClosing` : se produit lorsqu'on tente de fermer la fenêtre
- `Move` : se produit lorsque la fenêtre est déplacée
- `Resize` : Se produit lorsque la fenêtre est redimensionnée

Windows Forms : System.Windows.Forms

La classe Form

❑ Exemple

```
using System;
using System.Drawing;
using System.Windows.Forms;
namespace test{
    public class MaForm : Form {
        public MaForm()
        {
            this.Size = new Size(300, 200);
            this.Text = "Ma lère Form" ;
        }

        public static void Main(string[] args)
        {
            Application.Run(new MaForm()); //démarré l'application avec sa
                                           //fenêtre principale
        }
    }
}
```

La classe `Control`

Définit la classe de base de tous les contrôles

□ Quelques propriétés :

- **Size** : permet de spécifier ou retrouver la taille du contrôle
- **Location** : obtient ou définit les coordonnées (de type `Point`) du coin gauche du contrôle par rapport au coin gauche de son conteneur
- **Visible** : permet de contrôler la visibilité du contrôle
- **Enabled** : permet de contrôler l'accessibilité au contrôle
- **Text** : permet de spécifier le texte du contrôle
- **BackgroundImage** : Obtient ou définit l'image (type `Image`) en arrière plan du contrôle
- **BackColor** / **ForeColor** : permet de spécifier la couleur d'arrière plan/de premier plan
- **Font** : gère la police du texte du contrôle

Windows Forms : `System.Windows.Forms`

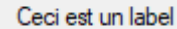
La classe `Control`

- **`Parent`** : obtient et définit le contrôle conteneur du contrôle
- **`ContextMenuStrip`** : obtient ou définit un menu contextuel associé au contrôle
- **`TabIndex`** : obtient ou définit l'ordre de tabulation du contrôle dans son conteneur
- **`ContainsFocus`** : teste si le contrôle ou un des contrôles qu'il contient possède le focus
- **`Tag`** : obtient ou définit un objet de n'importe quel type comme une propriété du contrôle
- **`Dock`** : obtient ou définit le bord (de type `DockStyle`) du conteneur dans lequel le contrôle sera mis (`DockStyle.Left`, `DockStyle.Right`, `DockStyle.Top`, `DockStyle.Bottom`, `DockStyle.None`, `DockStyle.Fill` (càd à tous les bords))
- **`Anchor`** : obtient ou définit avec quel bord du contrôle la distance doit rester constante par rapport à son conteneur (de type `AnchorStyle` : `AnchorStyle.Left`, `AnchorStyle.Right`, `AnchorStyle.Top`, `AnchorStyle.Bottom` (on peut les combiner avec l'opérateur `|`) ou `AnchorStyle.None`)

Windows Forms : System.Windows.Forms

Les contrôles

- **Contrôle Label**

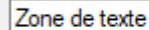


Le contrôle Label est dédié à l'affichage d'une étiquette texte

Voici ses propriétés importantes :

- **Text** permet de récupérer ou de définir le texte du label

- **Contrôle TextBox**



Le contrôle TextBox est dédié à la saisie d'un texte par l'utilisateur

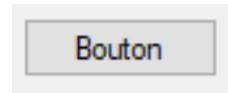
Voici ses propriétés importantes :

- **Text** permet de récupérer ou de définir la valeur saisie ;
- **Multiline** précise si le texte saisi peut comporter une ou plusieurs lignes ;
- **ReadOnly** permet, quand elle vaut true, d'en faire une zone en lecture seule (saisie impossible).
- L'évènement **TextChanged** est déclenché lors de la modification du texte de la zone de saisie.

Windows Forms : `System.Windows.Forms`

Les contrôles

- **Contrôle Button**



Le contrôle Button permet de créer un bouton.

Voici ses propriétés importantes :

- `Text` récupère ou définit le texte affiché dans le bouton ;
- `Enabled` active ou désactive le bouton (clic impossible) ;
- L'évènement `Click` permet de gérer le clic sur le bouton.

Windows Forms : System.Windows.Forms

Evènements sur les contrôles

- Un **évènement** est une action utilisateur (clic, saisie clavier, etc.).
- Chaque objet graphique contient ses propres évènements sous forme de propriétés de type **Event**
- Un **gestionnaire d'évènement** est une méthode qui réagit à cet évènement.
- On délègue le traitement d'un évènement à un gestionnaire d'évènement à l'aide de l'opérateur **+=**

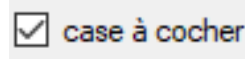
- **Exemple:**

```
Button b1= new Button();  
// Déléguer le traitement de l'évènement Click:  
b1.Click += b1_Click;  
...  
  
// Gestionnaire de l'évènement Click:  
private void      b1_Click(object sender, EventArgs e) {  
    MessageBox.Show("Bouton cliqué !");  
}
```

Windows Forms : `System.Windows.Forms`

Les contrôles

- **Contrôle `CheckBox`**



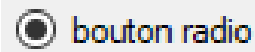
Le contrôle `CheckBox` permet de créer des cases à cocher (options cumulatives)

Voici ses propriétés importantes :

- `Checked` récupère ou définit l'état de la case (cochée ou non) ;
- `Text` représente le label associé à la case à cocher.
- L'évènement `CheckedChanged` permet de gérer le changement d'état de la case à cocher.

Windows Forms : `System.Windows.Forms`

Les contrôles



- **Contrôle RadioButton**

Le contrôle RadioButton permet de créer un bouton radio.

Les radioboutons sont liés entre eux par leur conteneur, un seul radio bouton peut être coché à un instant donné.

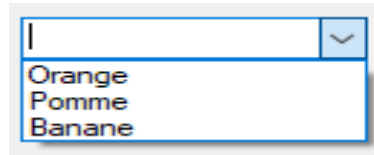
Voici ses propriétés importantes :

- **Checked** récupère ou définit l'état de la case (cochée ou non) ;
- **Text** représente le label associé au radio bouton.
- L'évènement **CheckedChanged** permet de gérer le changement d'état du radio bouton.

Windows Forms : `System.Windows.Forms`

Les contrôles

- **Contrôle ComboBox**



Le contrôle ComboBox définit une liste déroulante associée à une zone de texte

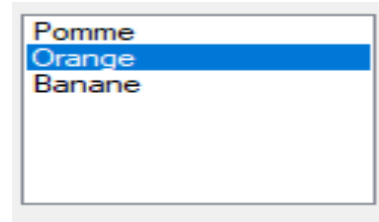
Voici ses propriétés importantes :

- **Items** regroupe ses valeurs sous la forme d'une liste (collection) d'objets
- **DropDownStyle** permet de choisir le style de la liste. Pour obtenir des valeurs non éditables par l'utilisateur, il faut choisir le style **DropDownList** ;
- **SelectedIndex** récupère ou définit l'indice de l'élément actuellement sélectionné. Le premier élément correspond à l'indice 0 ;
- **SelectedItem** renvoie l'élément actuellement sélectionné sous la forme d'un objet.
- L'évènement **SelectedIndexChanged** permet de gérer le changement de la valeur sélectionnée de la liste.

Windows Forms : System.Windows.Forms

Les contrôles

- **Contrôle ListBox**



Le contrôle ListBox permet de créer une liste d'éléments.

Voici ses propriétés importantes :

- **Items** regroupe ses valeurs sous la forme d'une liste (collection) d'objets
- **SelectionMode** permet de choisir si la liste est à sélection simple, multiple ou si la sélection est désactivée ;
- **SelectedIndex** récupère ou définit l'indice de l'élément actuellement sélectionné. Le premier élément correspond à l'indice 0 ;
- **SelectedItems** renvoie les éléments actuellement sélectionnés sous la forme d'une liste d'objets.
- L'évènement **SelectedIndexChanged** permet de gérer le changement de sélection dans la liste.

Windows Forms : `System.Windows.Forms`

Les contrôles

- **Contrôle `MenuStrip`**

- Le contrôle `MenuStrip` permet de créer une barre de menus déroulants.
- Une entrée de menu déroulant peut être une commande (`MenuItem`), une liste déroulante, une zone de texte ou un séparateur.
- L'évènement `Click` permet de gérer le clic sur une commande du menu déroulant.

Windows Forms : `System.Windows.Forms`

Les panneaux

- **Contrôle Panel**

- Permet de créer une zone du formulaire afin d'y introduire d'autres contrôles
- Il possède beaucoup de ressemblances avec le **GroupBox**
- Peut avoir une barre de défilement grâce à l'option **AutoScroll**

- **Propriétés**

- **AutoScroll** : Détermine si le Panel possède une barre de défilement lorsque les contrôles dépassent la limite visible du Panel.
- **BorderStyle** : Détermine l'apparence visuelle de la bordure du Panel: **None** pour aucun style de bordure, **FixedSingle** pour une simple bordure et **Fixed3D** pour une apparence 3D.
- **Cursor** : Indique l'apparence du curseur lorsque celui-ci survole le contrôle.
- **Visible** : Détermine si le contrôle est visible ou caché.

Windows Forms : `System.Windows.Forms`

Les panneaux

- **Contrôle `FlowLayoutPanel`**

- Spécialise le contrôle **`Panel`**
- Permet d'ajuster dynamiquement les contrôles (horizontalement ou verticalement) qu'il contient lorsque l'utilisateur fait un redimensionnement du formulaire

- **Propriétés**

- **`FlowDirection`**: sens d'insertion des contrôles dans le **`FlowLayoutPanel`**: **`LeftToRight`**, **`TopDown`**, **`RightToLeft`** et **`BottomUp`**
- **`WrapContent`**: Indique si le contenu du **`FlowLayoutPanel`** est encapsulé ou attaché à la limite du contrôle
- **`VerticalScroll`**: Définit les caractéristiques de la barre de défilement lors d'un affichage vertical.

Windows Forms : `System.Windows.Forms`

Les panneaux

- **Contrôle `TableLayoutPanel`**

- Spécialise le contrôle **`Panel`**
- Contient un tableau formé de lignes et de colonnes où les cellules contiennent des contrôles

- **Propriétés**

- **`CellBorderStyle`**: définit le style de bordure des cellules, `None` par défaut, donc pas de bordure
- **`ColumnCount`**: Indique le nombre de colonnes.
- **`Controls`**: Permet d'accéder à tous les contrôles du panneau
- **`ColumnStyles`** et **`RowStyles`**: définit les style des colonnes et des lignes
- **`GrowStyle`**: définit l'agrandissement du contrôle lorsqu'on ajoute un nouveau contrôle.
- **`RowCount`**: Indique le nombre de lignes

Windows Forms : `System.Windows.Forms`

Les panneaux

- **Contrôle `SplitContainer`**

- Crée une zone de deux panneaux séparés par une barre mobile : le *Splitter*

- **Propriétés**

- **`Orientation`** : spécifie l'orientation des panneaux, `Vertical` (par défaut) ou `Horizontal`.
 - **`Panel1`** : Affiche les différentes propriétés du `Panel1`.
 - **`Panel2`** : Affiche les différentes propriétés du `Panel2`.
 - **`Panel1Collapsed`** : Détermine si le `Panel1` est réduit ou développé.
 - **`Panel2Collapsed`** : Détermine si le `Panel2` est réduit ou développé.
 - **`SplitterWidth`** : Détermine la largeur du `Splitter`.

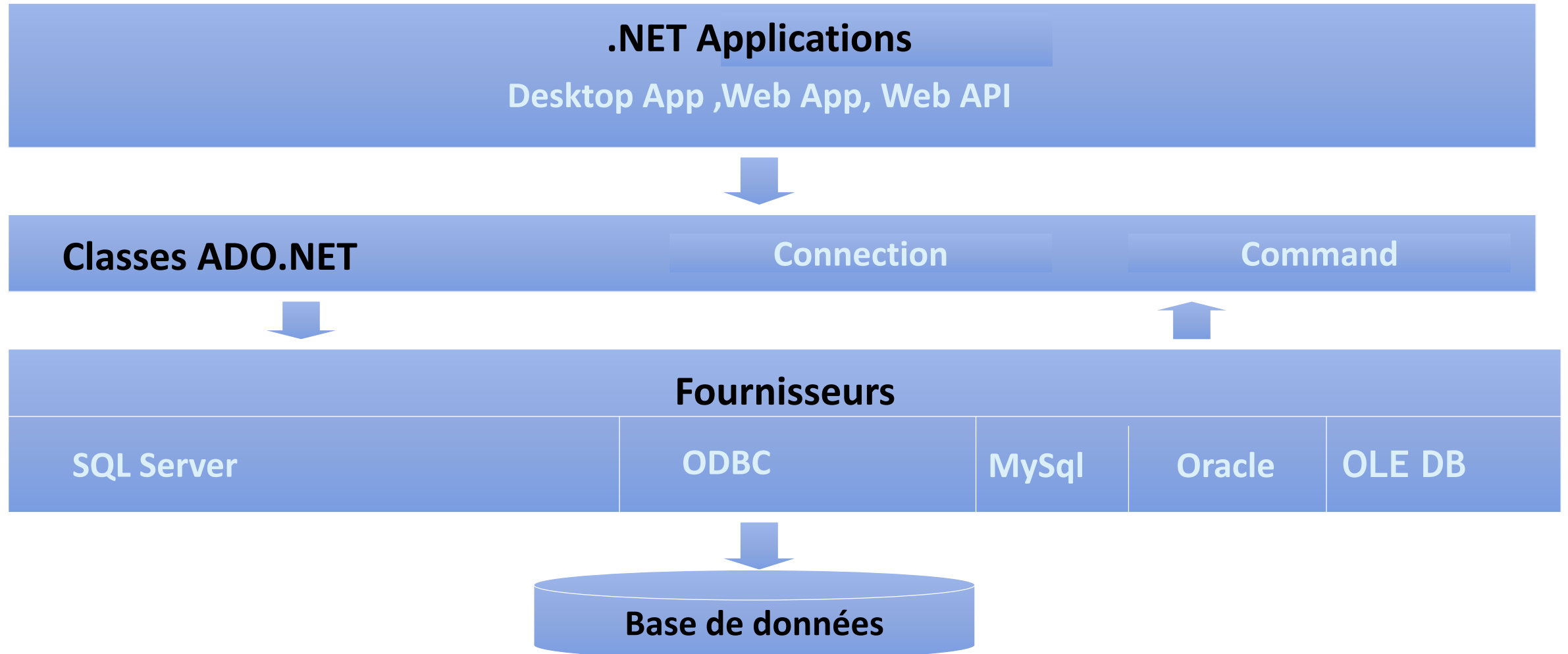


Gestion des bases de de données Technologie ADO.NET

ADO.NET

- ADO .NET (ActiveX Data Object) est une collection de classes et d'interfaces permettant l'accès à des données relationnelles, des données hiérarchiques, des données XML ou des données typées
- L'architecture ADO.NET est orientée vers les architectures réseaux et supporte le mode déconnecté
- Il contient des fonctionnalités concernant la manipulation des données à l'aide du langage LINQ
- ADO.NET est conçu pour communiquer avec plusieurs fournisseurs de bases de données
- Le namespace principal est Microsoft.Data (ancien System.Data)

Architecture ADO.NET



Fournisseurs (Providers)

Les providers fournis en standard dans le framework .NET sont :

- SQL Server (.NET SQL Server Data Provider)
- ODBC (.NET ODBC Data Provider)
- Oracle (.NET Oracle Data Provider)
- MySql (.NET MySql Data Provider)
- OLEDB (.NET OLEDB Data Provider)

Fournisseurs (Providers)

Chaque fournisseur nécessite quatre classes décrites brièvement dans le tableau ci-dessous :

Nom de la classe	Description
Connection	Réalise la connexion à la base de données.
Command	Exécute des requêtes SQL ou des procédures stockées.
DataReader	Récupère un flux de données en lecture seule, et en avant seulement.

Namespaces utiles

System.Data	Comprend les classes qui constituent l'architecture ADO.NET permettant de construire des composants qui gèrent les données provenant de plusieurs sources
System.Data.Common	Contient des classes partagées par les fournisseurs de données .NET Framework qui décrivent une collection de classes utilisées pour accéder à une source de données
System.Data.OleDb	Contient des classes qui constituent le fournisseur de données .NET Framework pour les sources de données compatibles OLE DB
System.Data.SqlClient	Contient des classes qui constituent le fournisseur de données .NET pour SQL Server
System.Data.Odbc	Contient des classes qui constituent le fournisseur de données .NET pour les sources de données ODBC
System.Data.OracleClient	Contient des classes qui constituent le fournisseur de données .NET pour les sources de données Oracle
System.Data.MySqlClient	Contient des classes qui constituent le fournisseur de données .NET pour les sources de données MySql

Connection à la base de données

- Permet de créer une connexion à une source ou fournisseur de donnée, elle représente une session unique vers une base de donnée.
- ADO.Net dispose de 4 type de classes Connection selon le fournisseur:
 - `SqlConnection` : pour l'accès au base de données SQL Server
 - `OdbcConnection` : pour l'accès au base de donnée de type ODBC
 - `OracleConnection` : pour l'accès au base de données Oracle
 - `MySqlConnection` : pour l'accès au base de données MySql
 - `OleDbConnection` : pour l'accès au donnée de type OLEDB

Connection à la base de données

Propriété ou méthode	Description
ConnectionString	Propriété de type String précisant tous les paramètres de la connexion
Open	Ouvre la connexion
Close	Ferme la connexion
IsOpen	Permet de tester si la connexion est ouverte

Requêtes sur la base de données

- La classe **Command** permet d'exécuter des requêtes SQL, ou des procédures stockées avec ou sans paramètres.
- Les commandes exécutées peuvent ou non retourner des résultats

Fournisseur managé	Nom de la classe Command
SQL Server	SqlCommand
ODBC	OdbcCommand
Oracle	OracleCommand
Mysql	MySqlCommand
OLEDB	OleDbCommand

■ Requête sur la base de données

Propriété ou méthode	Description
Connection	Objet Connection représentant la connexion sur laquelle est exécutée la requête SQL
CommandType	Type de la commande : Text, StoredProcedure, TableDirect.
CommandText	Commande SQL, ou nom de la table ou de la procédure, suivant la valeur de CommandType.
ExecuteNonQuery	Exécute une requête de mise à jour
ExecuteReader	Exécute la commande, et construit un objet de type DataReader utilisé pour parcourir les données.
ExecuteScalar	Exécute une requête qui retourne une seule valeur généralement d'agrégation comme count, sum, avg, max,

Requêtes sur la base de données

La classe *DataReader* est utilisée pour accéder à des données provenant d'une source de données ,elle permet de lire des données en lecture seule, en avant seulement, et de façon optimisée et elle reste en permanence connecté à la source de données.



Fournisseur managé	Nom de la classe DataReader
SQL Server	<i>SqlDataReader</i>
OLEDB	<i>OleDbDataReader</i>
ODBC	<i>OdbcDataReader</i>
Oracle	<i>OracleDataReader</i>
MySql	<i>MySqlDataReader</i>

Requêtes sur la base de données

Propriété ou méthode	Description
FieldCount	Nombre de colonnes de la ligne courante
Read	Avance le DataReader au tuple suivant
L'indexeur ["nomcol"]	Retourne la valeur de la colonne indiquée, on peut aussi utiliser des fonctions Get avec le type adéquat
Close	Ferme le DataReader.

Exemple avec SQLServer

```
SqlConnection conn = new SqlConnection();
SqlCommand cmd = new SqlCommand();
SqlDataReader reader;
conn.ConnectionString = @"server=(localdb)\mssqllocaldb;database=mydb;"
conn.Open();
cmd.Connection = conn;
cmd.CommandText = @"Select firstname, lastname from employee
                    where lastname like 'a%'";
reader = cmd.ExecuteReader();
while (reader.Read())
{
    Console.WriteLine(reader.GetString(0)); //equivalent à reader[0]
    Console.WriteLine(reader.GetString(1)); //equivalent à reader[1]
}
reader.Close();
cmd.CommandText = "Select count(*) from employee";
int nbEmployes =(int) cmd.ExecuteScalar();
Console.WriteLine("Nombre d'employes " + nbEmployes);
```

Exemple avec ODBC (SGBD quelconque)

```
//il faut créer une source de données ODBC database
OdbcConnection con=new OdbcConnection();
OdbcCommand command=new OdbcCommand();
con.ConnectionString = "dsn=mydb";
con.Open();
command.Connection = con;
command.CommandText = "select * from employe";
OdbcDataReader reader= command.ExecuteReader();
while (reader.Read())
{
    Console.WriteLine(reader["firstname"]);
    Console.WriteLine(reader["lastname"]);
}
reader.Close();
command.CommandText = "Select count(*) from employe";
long nbEmployes = (long)command.ExecuteScalar();
Console.WriteLine("Nombre d'employes " + nbEmployes);
con.Close();
```

Exemple avec MySql

```
MySqlConnection con = new MySqlConnection();
MySqlCommand command = new MySqlCommand();
con.ConnectionString= @"Server=localhost;Database=mydb;
                        Uid=abder;Pwd=abder;";

con.Open();
command.Connection = con;
command.CommandText = "select * from employe";

MySqlDataReader reader = command.ExecuteReader();
while (reader.Read())
{
    Console.WriteLine(reader["firstname"]);
    Console.WriteLine(reader["lastname"]);
}
reader.Close();
command.CommandText = "Select count(*) from employe";
long nbEmployes = (long)command.ExecuteScalar();
Console.WriteLine("Nombre d'employes " + nbEmployes);
con.Close();
```